

## Unit 11: Generic Programming with Templates

### CONTENTS

Objectives

Introduction

11.1 Class Template

11.2 Class Templates with Multiple Parameter

11.3 Function Template

11.4 The Typename Keyword

11.5 Template Specialization

11.6 Function Templates with Multiple Parameters

11.7 Point of Instantiation

11.8 Overloading of Template Function

Summary

Keywords

Self Assessment

Review Questions

Answers for Self Assessment

Further Readings

### Objectives

After studying this unit, you will be able to:

- Describe the function of templates
- Describe the classes of templates
- Understand generic programming

### Introduction

One of C++'s most advanced and powerful features is the template. Despite the fact that it was not included in the original C++ specification, it was introduced several years ago and is now supported by all recent C++ compilers. It is possible to create generic functions and classes using templates. The kind of data on which a generic function or class is based. This allows the function or class to work with a variety of data types without having to explicitly recode specialised versions for each data type.

A template can be considered as a kind of macro. When an object of a specific type is defined for actual use, the template definition for that class is substituted with the required data type. Since a template is defined with a parameter that would be replaced by a specified data type at the time of actual use of the class or function, the templates are sometimes called parameterized classes or functions.



Did you know?

What is a template parameter?

A template parameter is a special kind of parameter that can be used to pass a type as argument: just like regular function parameters can be used to pass values to a function, template parameters allow to pass values and also types to a function.

## 11.1 Class Template

- Like function templates, we can also create class templates for generic class operations.
- A class implementation that is same for all classes, only the data types used are different.
- Generally, we need to create a different class for each data type OR create different member variables and functions within a single class.

### Create object of class template

To create a class template object, you need to define the data type inside a < > when creation.

Syntax:-

```
className<dataType> classObject;
```



Example:

```
className<int> classObject;
```

```
className<double> classObject;
```



Lab Exercise

```
#include<iostream>
using namespace std;
template<class T>
class sum{
    T num1,num2;
public:
    sum(T a, T b){
        num1=a;
        num2=b;
    }
    void disp(){
        cout<<"Numbers are "<<num1<< " "<<num2<<endl;
        cout<<"sum is"<<add()<<endl;
    }
    T add(){return num1+num2;};
};
main(){
    sum<int> sumint(50,50);
    sum<float> sumfl(50.25,15.225);
    cout<<"Int Result" <<endl;
    sumint.disp();
    cout<<"Float Result"<<endl;
    sumfl.disp();
}
```

Output

```

Int Result
Numbers are 50  50
sum is100
Float Result
Numbers are 50.25  15.225
sum is65.475

Process returned 0 (0x0)   execution time : 0.144 s
Press any key to continue.

```

## 11.2 Class Templates with Multiple Parameter

We can use more than one generic data type in a class template. They are declared as a comma-separated list within the template specification as shown below:

```

template<class T1, class T2, ...>
class classname
{
    .....
    .....    (Body of the class)
    .....
};

```



Lab Exercise

```

#include <iostream>

using namespace std;

template<class T1, class T2>
class Test
{
    T1 a;
    T2 b;
public:
    Test(T1 x, T2 y)
    {
        a = x;
        b = y;
    }
    void show()
    {
        cout << a << " and " << b << "\n";
    }
};

int main()
{
    Test <float,int> test1 (1.23,123);
    Test <int,char> test2 (100,'W');

    test1.show();
    test2.show();

    return 0;
};

```

Output

1.23 and 123

100 and W

### 11.3 Function Template

In C++ when a function is overloaded, many copies of it have to be created, one for each data type it acts on. In the example of the `max()` function, which returns the greater of the two values passed to it this function would have to be coded for every data type being used. Thus, you will end up coding the same function for each of the types, like `int`, `float`, `char`, and `double`. A few versions of `max()` are:

```
Int max ( int x, int y)
{
return x > y ? x : y
}

char max ( char x, char y )
{
return x > y ? x : y
}

double max (double x , double y)
{
return x > y ? x : y
}

float max ( float x, float y)
{
return x > y ? x : y
}
```

Here you can see, the body of each version of the function is identical. The same code has to be repeated to carry out the same function on different data types. This is a waste of time and effort, which can be avoided using the template utility provided by C++.

“A template function may be defined as an unbounded functions “all the possible parameters to the function are not known in advance and a copy of the function has to be created as and when necessary. Template functions are using the keyword, `template`. Templates are blueprints of a function that can be applied to different data types.

Syntax of Template

Template < class type 1, type 2 ... >

Void function - name ( type 2 parameter 1, type 1 parameter 2 ) {...}



Example

Template < class X >

X min ( X a , X b )

```
{
return (a < b) ? a : b ;
}
```

---

Unit 11: Generic Programming with Templates

---

This list of parameter types is called the formal parameter list of the template, and it cannot be empty. Each formal parameter consists of the keyword, type name, followed by an identifier. The identifier can be built-in or user-defined data type, or the identifier type. When the function is invoked with actual parameters, the identifier type is substituted with the actual type of the parameter. This allows the use of any data type. The template declaration immediately precedes the definition of the function for which the template is being defined. The template declaration, followed by the function definition, constitutes the template definition. The template of the max ( ) function is coded below:

```
# include <iostream.h>

template < class type >

type max ( type x , type y)
{
return x > y ? x : y ;
}

int main ( )
{
cout << " max ( 'A' , 'a') : " << max ( 'A' , 'a') << endl ;
cout << " max ( 30 , 40 ) : " << max ( 30 , 40 ) << endl;
cout << " max ( 45 . 67F, 12 . 32 F ) : " << max (45. 67F, 12 .
32 F) << endl;
return 0 ;
}
```

## Output

```
max ( 'A' , 'a') : a
max ( 30 , 40 ) : 40
max ( 45.67F, 12 . 32F ) : 45 . 67
```

In the example, the list of parameters is composed of only one parameter. The next line specifies that the function takes two arguments and returns a value, all of the defined in the formal parameter list. See what happens if the following command is issued, keeping in mind the template definition for max ( ) :

```
max ( a , b ) ;
```

in which a and b are integer type variables. When the user invokes max ( ) , using two int values, the identifier, 'type', is substituted with int, wherever it is present. Now max ( ) works just like the function int max (int, int) defined earlier, to compare two int values. Similarly, if max ( ) is invoked using two double values, 'type' is replaced with 'double'. This process of substitution, depending on the parameter type passed to the function, is called template instantiation. The template specifies how individual functions will be constructed, given a set of actual types. The template facility allows the creation of a blueprint for a function like max ( ).



## Example

```
# include <iostream.h>

template < class type >

type square ( type a)
{
type b;
```

```

b= a*a ;
return b.;
}
int main ( )
{
cout << " square (25 ) : " << square ( 25) << endl;
cout << " square 40 : " << square 40 << endl;
return 0 ;
}

```

Output

Square ( 25 . 45F) : 1125

Square 90 : 1600

Here is another example of the use of template function. Consider the following classes:

```

class IRON
{
private :
float density ;
public :
IRON ( ) {density = 8.9 }
Float Density ( ) { return density ;}
};

```



#### Lab Exercise

```

#include <iostream>
using namespace std;
template <typename T>
T add(T num1, T num2) {
    return (num1 + num2);
}
int main() {
    int result1;
    double result2;
    result1 = add<int>(50, 100);
    cout << "Sum of integers = " << result1 << endl;
    result2 = add<double>(10.2, 9.25);
    cout << "Sum of double = " << result2 << endl;
    return 0;
}

```

Output

```
Sum of integers = 150
Sum of double = 19.45

Process returned 0 (0x0)   execution time : 0.015 s
Press any key to continue.
```



Did you know?

Is the template or blueprint can then be instantiated for all data types?

Yes, the template or blueprint can then be instantiated for all data types, eliminating duplication of the source code. The identifier, 'type', can be used within the function body of a function that, otherwise, remains unchanged.

## 11.4 The Typename Keyword

The keywords typename and class can be freely interchanged. For example

Template < class T >

Int maxindex ( T arr [ ], int size )

```
{
...
}
```

## 11.5 Template Specialization

While template functions and classes are usable for any data type, that would hold true only, as long as the body of the functions or the class is identical throughout. So, if you have,

Template < typename T .

Void swap ( T + lhs , T + rhs )

```
{
T tmp ( + lhs ) ;
+ lhs = + rhs ;
+ rhs = tmp ;
}
```

the previous template will be instantiated correctly for any of the pointer types passed to it -except for the data types char + , unsigned char + , and signed char + . Since, the body of the function.



Caution:

Differs for these types, you would use template specialize the template function.

Specialization would consist of instantiating the template definition for a specific data type. So, when the compiler needs to instantiate the template, it finds a predefined version already existing and uses it. For example, just after the previous template declaration, you could create a template specialization for char + like this:

```

Template e >
Void swap < char > ( char + lhs , char + rhs )
{
char + tmp = new char [ strlen (lhs) + 1 ];
strcpy ( tmp , lhs) .;
strcpy ( tmp , lhs);
Strcpy ( lhs , rhs) ; Notes
Strcpy ( rhs , tmp ) ;
}

```

## 11.6 Function Templates with Multiple Parameters

Like template classes, we can use more than one generic data type in the template statement, using a comma-separated list as shown below:

```

template<class T1, class T2, ...>
returntype functionname(arguments of types T1, T2,...)
{
    .....
    ..... (Body of function)
    .....
}

```



Example

```

#include<iostream>
#include<string>
using namespace std;
template<class T1,class T2>
void display(T1 x,T2 y)
{
cout<<x<<" "<<y<<"\n";
}
int main(){
display(858,"ABD");
display(858.58, 1024);
return 0;
}

```

Output

858 ABD

858.58 1024

## 11.7 Point of Instantiation

A template function gets instantiated under the following circumstances:

1. Implicitly instantiated because it is referenced from a function call that depends on a template argument.
2. Implicitly instantiated because it is referenced within a default argument in a declaration.
3. The point of instantiation of a function template specialization immediately follows the declaration or definition that refers to the specialization.

## 11.8 Overloading of Template Function

A template function may be overloaded either by template functions or ordinary functions of its name. In such cases, the overloading resolution is accomplished as follows:

1. Call an ordinary function that has an exact match.
2. Call a template function that could be created with an exact match.
3. Try normal overloading resolution to ordinary functions and call the one that matches.

An error is generated if no match is found. Note that no automatic conversions are applied to arguments on the template functions. Program 12.8 shows how a template function is overloaded with an explicit function.



Notes

- Overloading a function template means having different sets of function templates which differ in their parameter list.
- If the function template is with the ordinary template, the name of the function remains the same but the number of parameters differs.



Example:

```
#include <iostream>
using namespace std;
template <class T>
void display(T t1)
{
    cout << "Display template value: "
         << t1 << "\n";
}
void display(int t1)
{
    cout << "With integer argument: "
         << t1 << "\n";
}
int main()
{
    display(100);
    display(10.2);
}
```

```

    display('A');
    return 0;
}

```

Output

```

With integer argument: 100
Display template value: 10.2
Display template value: A

Process returned 0 (0x0)   execution time : 0.045 s
Press any key to continue.

```



#### Task

Write a program to demonstrate working of function template in C++.

Write a program to demonstrate working of overloading function template in C++.

We have seen that a template can have multiple arguments. It is also possible to use non-type arguments. That is, in addition to the type argument *T*, we can also use other arguments such as strings, function names, constant expressions and built-in types. Consider the following example:

```

template<class T, int size>
class array I a[size];
{
    T a[size];           // automatic array initialization
    //.....
    //.....
}

```

This template supplies the size of the array as an argument. This implies that the size of the array is known to the compiler at the compile time itself. The arguments must be specified whenever a template class is created. Example:

```

array<int,10>    a1;           // Array of 10 integers
array<float,5>   a2;           // Array of 5 floats
array<char,20>   a3;           // String of size 20

```

The size is given as an argument to the template class.

### Summary

- C++ supports a mechanism known as template to implement the concept of generic programming.
- Templates classes may be defined as the layout and operations for an unbounded set of related classes.
- Templates allows us to generate a family of classes or a family of functions to handle different data types.

---

**Unit 11: Generic Programming with Templates**

---

- Template classes and functions eliminate code duplication for different types and thus make the program development easier and more manageable.
- We can use multiple parameters in both the class templates and function templates. A specific class created from a class template is called a template class and the process of creating a template class is known as instantiation. Similarly, a specific function created from a function template is called a template function.
- Like other functions, template functions can be overloaded.
- Member functions of a class template must be defined as function templates using the parameters of the class template.
- While template functions and classes are usable for any data type, that would hold true only, as long as the body of functions or the class is identical throughout.
- We may also use non-type parameters such basic or derived data types as arguments templates.

**Keywords**

**Template:** A template function may be defined as an unbounded functions. All the possible parameters to the function are not known in advance and a copy of the function has to be created as and when necessary.

**Template Classes:** Template classes may be defined as the layout and operations for an unbounded set of related classes.

**Function Template:** We write a generic function that can be used for different data types. Function templates are special functions that can operate with generic types.

**Generic Programming in C++:** Generic programming is a technique where generic types are used as parameters in algorithms so that they can work for a variety of data types.

**Self Assessment**

1. What is meant by the template parameter?
  - A. It can be used to pass a type as an argument
  - B. It can be used to evaluate a type
  - C. It can of no return type
  - D. It can be used to delete a type
2. Which keyword can be used in template?
  - A. class
  - B. typename
  - C. both class & typename
  - D. function
3. What is a template?
  - A. A template is a formula for creating a generic class
  - B. A template is used to manipulate the class
  - C. A template is used for creating the attributes
  - D. None of the above
4. What is the validity of template parameters?

- A. inside that block only
  - B. inside the class
  - C. whole program
  - D. inside the main class
5. Which of the following best defines the syntax for template function?
- A. `template return_type Function_Name(Parameters)`
  - B. `template return_type Function_Name(Parameters)`
  - C. All of above
  - D. None of the above
6. From where does the template class derived?
- A. Regular non-templated C++ class
  - B. Templated class
  - C. All of above
  - D. None of the above
7. Which of the following is used for generic programming?
- a. Virtual functions
  - b. Modules
  - c. Templates
  - d. Abstract Classes
8. Which of the following is correct about templates?
- A. It is a type of compile time polymorphism
  - B. It allows the programmer to write one code for all data types
  - C. Helps in generic programming
  - D. All of the mentioned
9. What is the difference between normal function and template function?
- A. The normal function works with any data types whereas template function works with specific types only
  - B. Template function works with any data types whereas normal function works with specific types only
  - C. Unlike a normal function, the template function accepts a single parameter
  - D. Unlike the template function, the normal function accepts more than one parameters
10. What does this template function indicates?
- ```
template<class T>
T func(T a)
{
    cout<<a;
}
```
- A. A function taking a single generic parameter and returning a generic type
  - B. A function taking a single generic parameter and returning nothing
  - C. A function taking single int parameter and returning a generic type
  - D. A function taking a single generic parameter and returning a specific non-void type

---

**Unit 11: Generic Programming with Templates**

---

11. Can we have overloading of the function templates?
  - A. Yes
  - B. No
  - C. May Be
  - D. Can't Say
12. What is a function template?
  - A. creating a function without having to specify the exact type
  - B. creating a function with having an exact type
  - C. creating a function without having blank spaces
  - D. creating a function without class
13. Which keyword is used for the template?
  - A. Template
  - B. template
  - C. Temp
  - D. temp
14. Which of the following is correct about templates?
  - A. It is a type of compile time polymorphism
  - B. It allows the programmer to write one code for all data types
  - C. Helps in generic programming
  - D. All of the mentioned
15. Pick out the correct statement.
  - A. you only need to write one function, and it will work with many different types
  - B. it will take a long time to execute
  - C. Duplicate code is increased
  - D. it will take a long time to execute & duplicate code is increased

**Review Questions**

1. What is generic programming? How is it implemented in C++?
2. A template can be considered as a kind of macro. Then, what is the difference between them? Distinguish between overloaded functions and function templates.
3. Distinguish between the terms class template and template class.
4. A class (or function) template is known as a parameterized class (or function). Comment.
5. What are Templates?
6. Define template instantiation.
7. List down the rules for using templates.
8. How can you get a template function instantiated?
9. Define the template class with the explaining program.
10. When can a template function be instantiated?

**Answers for Self Assessment**

- |       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. A  | 2. C  | 3. A  | 4. A  | 5. C  |
| 6. C  | 7. C  | 8. D  | 9. B  | 10. A |
| 11. A | 12. A | 13. B | 14. C | 15. A |

**Further Readings**

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

**Web Links**

<https://www.codecademy.com/learn/learn-c-plus-plus>

<https://isocpp.org/wiki/faq/templates>

<https://www.startertutorials.com/blog/generic-programming-cpp.html>